

Relatorio do trial

PARTICIPANTE

Luis Henrique Gonçalves Barbosa

KATA

Tarifador de Hospedagem e Pousada

TRATAMENTO

Com assistente de IA

TEMPO FINAL

01:05

TEMPO ESGOTADO

Nao

INICIO

16/09/2026, 13:03:14

FIM

16/09/2026, 13:04:19

RESULTADO E OBSERVACOES

8 de 8 testes unitários de aceitação passaram com sucesso (100%).

ARQUIVOS ANEXADOS

3 arquivo(s) de código ou texto anexado(s) ao trial.

- TarifadorHospedagem.java (67 linhas)
- CategoriaQuarto.java (26 linhas)
- texto-do-trial.txt (67 linhas)

TarifadorHospedagem.java

```
package br.ufc.lab02.hospedagem;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

public class TarifadorHospedagem {

    private static final BigDecimal TARIFA_CURTA = BigDecimal.valueOf(160.00);
    private static final BigDecimal TARIFA_MEDIA = BigDecimal.valueOf(130.00);
    private static final BigDecimal TARIFA_LONGA = BigDecimal.valueOf(100.00);
    private static final BigDecimal VALOR_HOSPEDE_EXTRA = BigDecimal.valueOf(40.00);
    private static final BigDecimal FATOR_DESCONTO_CONVENIO = BigDecimal.valueOf(0.90);

    public BigDecimal calcularHospedagem(LocalDate checkIn, LocalDate checkOut, int
numeroHospedes, CategoriaQuarto categoria, boolean possuiConvenio) {
        if (checkIn == null || checkOut == null) {
            throw new IllegalArgumentException("Datas de check-in e check-out sao
obrigatorias.");
        }
        if (!checkOut.isAfter(checkIn)) {
            throw new IllegalArgumentException("A data de check-out deve ser posterior ao
check-in.");
        }
        if (numeroHospedes <= 0 || numeroHospedes > 5) {
            throw new IllegalArgumentException("Numero de hospedes deve ser entre 1 e
5.");
        }
        if (categoria == null) {
            throw new IllegalArgumentException("Categoria do quarto e obrigatoria.");
        }

        long totalDiarias = ChronoUnit.DAYS.between(checkIn, checkOut);

        // 1. Tarifa base escalonada por diária
        BigDecimal tarifaDiaria;
        if (totalDiarias <= 3) {
            tarifaDiaria = TARIFA_CURTA;
        } else if (totalDiarias <= 7) {
            tarifaDiaria = TARIFA_MEDIA;
        } else {
            tarifaDiaria = TARIFA_LONGA;
        }
        BigDecimal custoBaseDiarias =
tarifaDiaria.multiply(BigDecimal.valueOf(totalDiarias));

        // 2. Adicional por hóspedes extras (acima de 2 hóspedes)
        BigDecimal custoHospedesExtras = BigDecimal.ZERO;
        if (numeroHospedes > 2) {
            int extras = numeroHospedes - 2;
            custoHospedesExtras = VALOR_HOSPEDE_EXTRA
                .multiply(BigDecimal.valueOf(extras))
                .multiply(BigDecimal.valueOf(totalDiarias));
        }

        // 3. Subtotal das diárias com multiplicador de categoria
        BigDecimal subtotalDiarias = custoBaseDiarias.add(custoHospedesExtras);
```

```
        BigDecimal subtotalComCategoria =
subtotalDiarias.multiply(BigDecimal.valueOf(categoria.getFatorMultiplicador()));

        // 4. Taxa fixa de higienização
        BigDecimal totalComHigienizacao =
subtotalComCategoria.add(categoria.getTaxaLimpeza());

        // 5. Desconto de convênio
        BigDecimal totalFinal = possuiConvenio
            ? totalComHigienizacao.multiply(FATOR_DESCONTO_CONVENIO)
            : totalComHigienizacao;

        return totalFinal.setScale(2, RoundingMode.HALF_EVEN);
    }
}
```

CategoriaQuarto.java

```
package br.ufc.lab02.hospedagem;

import java.math.BigDecimal;

public enum CategoriaQuarto {
    STANDARD(1.00, BigDecimal.valueOf(50.00)),
    LUXO(1.30, BigDecimal.valueOf(80.00)),
    PRESIDENCIAL(1.70, BigDecimal.valueOf(120.00));

    private final double fatorMultiplicador;
    private final BigDecimal taxaLimpeza;

    CategoriaQuarto(double fatorMultiplicador, BigDecimal taxaLimpeza) {
        this.fatorMultiplicador = fatorMultiplicador;
        this.taxaLimpeza = taxaLimpeza;
    }

    public double getFatorMultiplicador() {
        return fatorMultiplicador;
    }

    public BigDecimal getTaxaLimpeza() {
        return taxaLimpeza;
    }
}
```

texto-do-trial.txt

```
package br.ufc.lab02.hospedagem;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

public class TarifadorHospedagem {

    private static final BigDecimal TARIFA_CURTA = BigDecimal.valueOf(160.00);
    private static final BigDecimal TARIFA_MEDIA = BigDecimal.valueOf(130.00);
    private static final BigDecimal TARIFA_LONGA = BigDecimal.valueOf(100.00);
    private static final BigDecimal VALOR_HOSPEDE_EXTRA = BigDecimal.valueOf(40.00);
    private static final BigDecimal FATOR_DESCONTO_CONVENIO = BigDecimal.valueOf(0.90);

    public BigDecimal calcularHospedagem(LocalDate checkIn, LocalDate checkOut, int
numeroHospedes, CategoriaQuarto categoria, boolean possuiConvenio) {
        if (checkIn == null || checkOut == null) {
            throw new IllegalArgumentException("Datas de check-in e check-out sao
obrigatorias.");
        }
        if (!checkOut.isAfter(checkIn)) {
            throw new IllegalArgumentException("A data de check-out deve ser posterior ao
check-in.");
        }
        if (numeroHospedes <= 0 || numeroHospedes > 5) {
            throw new IllegalArgumentException("Numero de hospedes deve ser entre 1 e
5.");
        }
        if (categoria == null) {
            throw new IllegalArgumentException("Categoria do quarto e obrigatoria.");
        }

        long totalDiarias = ChronoUnit.DAYS.between(checkIn, checkOut);

        // 1. Tarifa base escalonada por diária
        BigDecimal tarifaDiaria;
        if (totalDiarias <= 3) {
            tarifaDiaria = TARIFA_CURTA;
        } else if (totalDiarias <= 7) {
            tarifaDiaria = TARIFA_MEDIA;
        } else {
            tarifaDiaria = TARIFA_LONGA;
        }
        BigDecimal custoBaseDiarias =
tarifaDiaria.multiply(BigDecimal.valueOf(totalDiarias));

        // 2. Adicional por hóspedes extras (acima de 2 hóspedes)
        BigDecimal custoHospedesExtras = BigDecimal.ZERO;
        if (numeroHospedes > 2) {
            int extras = numeroHospedes - 2;
            custoHospedesExtras = VALOR_HOSPEDE_EXTRA
                .multiply(BigDecimal.valueOf(extras))
                .multiply(BigDecimal.valueOf(totalDiarias));
        }

        // 3. Subtotal das diárias com multiplicador de categoria
        BigDecimal subtotalDiarias = custoBaseDiarias.add(custoHospedesExtras);
```

```
        BigDecimal subtotalComCategoria =
subtotalDiarias.multiply(BigDecimal.valueOf(categoria.getFatorMultiplicador()));

        // 4. Taxa fixa de higienização
        BigDecimal totalComHigienizacao =
subtotalComCategoria.add(categoria.getTaxaLimpeza());

        // 5. Desconto de convênio
        BigDecimal totalFinal = possuiConvenio
            ? totalComHigienizacao.multiply(FATOR_DESCONTO_CONVENIO)
            : totalComHigienizacao;

        return totalFinal.setScale(2, RoundingMode.HALF_EVEN);
    }
}
```